Greenhouse Sentinel

Product handbook and docs-as-code portfolio demonstration

Fictional product · Real documentation workflow · Katie Kearns · 2026

This public sample contains no customer, proprietary, export-controlled, or classified information.

Greenhouse Sentinel

A fictional product with real documentation engineering behind it.

Greenhouse Sentinel is a small controller that watches three environmental signals and recommends a response before plants are stressed. This site documents the fictional controller and, just as importantly, the docs-as-code system that publishes this site.

Start the product tour

See how the repo works

One source, several useful outputs

Authors write accessible Markdown. Automated checks verify navigation, internal links, anchors, and basic authoring rules. Zensical turns the reviewed source into this searchable site, and the export job creates Word and PDF versions for readers who need portable files.

The fictional product in 30 seconds

Greenhouse Sentinel samples temperature, humidity, and soil moisture every 60 seconds. A rule engine assigns the greenhouse a state: **normal**, **watch**, or **act**. The operator dashboard explains the reading and links each alert to a documented response.

Greenhouse Sentinel does not exist. Its behavior, data, and interface are intentionally simple and invented. The repository exists to demonstrate documentation architecture, authoring, validation, CI, publishing, and multi-format delivery without confidential information.

Explore both layers

- Use the quick start and operating guide as if you were evaluating product documentation.
- Read How this repository works to inspect the mechanics behind the site.
- Review the accessibility standard and release checklist to see how quality becomes part of the workflow.

Quick start

This walkthrough connects a fictional Sentinel hub and confirms that its three sensors are reporting. No hardware is required; the commands are illustrative.

Before you begin

You need a Sentinel hub, one temperature and humidity probe, one soil probe, and access to the greenhouse Wi-Fi network.

Connect and verify

1. Place the hub above the irrigation line and away from direct spray.
2. Connect both probes before applying power.
3. Join the temporary network named `sentinel-setup`.
4. Open `http://sentinel.local` and choose the greenhouse network.
5. Select **Run sensor check**.

A successful check reports all three readings and changes the hub indicator from amber to green.

```
temperature   24.1 C   ready
humidity      58 %     ready
soil          41 %     ready
```

Understand the first result

The initial state should be **normal**. The controller waits for three consecutive samples before changing state, which prevents a single noisy reading from creating an alert. See Decision states for the complete rule.

If a sensor remains unavailable, follow A sensor reports no data.

The link text names the reader's problem instead of saying “click here.” The validator rejects common context-free labels and verifies the stable troubleshooting anchor. This makes the reference clearer in the page, a screen reader's links list, Word, and PDF.

Next step

Continue to the daily operating routine.

Operating guide

The operating routine is designed for a volunteer who checks the greenhouse at opening and closing.

Daily check

1. Confirm the dashboard timestamp is less than two minutes old.
2. Review the current state and its explanation.
3. Compare soil moisture across the three most recent samples.
4. Acknowledge any completed response.
5. Record an observation only when local conditions differ from the sensor data.

Respond to a state

Normal

No intervention is required. Continue the scheduled check.

Watch

Inspect the greenhouse within 30 minutes. Confirm the sensor is unobstructed and compare the reading with a handheld instrument when one is available.

Act

Follow the response displayed with the alert. Ventilate for high temperature, inspect irrigation for low soil moisture, or check airflow for sustained high humidity. Never bypass a physical safety control.

Acknowledge an alert

Acknowledgement records that a person saw the alert; it does not change the sensor reading or decision state. Add a short note that states what you observed and what you did.

Good: `Opened roof vent; temperature fell to 27 C within 12 minutes.`

Avoid: `Fixed.`

For the system behavior behind these states, see Decision states.

This source link points to `architecture.md#decision-states`. The file path keeps the link useful in GitHub and other Markdown tools; Zensical rewrites it for the published site. The destination has an explicit `decision-states` anchor, so minor heading edits do not break the reference. The independent validator confirms that both the file and anchor exist before publication.

Architecture

Greenhouse Sentinel separates measurement, evaluation, presentation, and response guidance so each part can be tested and explained independently.

System boundary

The hub receives local sensor readings and produces a recommendation. It does not operate pumps, vents, heaters, locks, or other physical equipment. A person remains responsible for any physical action.

```
Sensors -> sample normalizer -> rule engine -> local event store -> dashboard
                                      |
                                      +-> response guidance
```

Components

- **Sample normalizer:** converts each supported sensor value to a consistent unit and marks stale data.
- **Rule engine:** evaluates the most recent valid samples against versioned thresholds.
- **Event store:** retains 30 days of readings, state changes, and acknowledgements on the hub.
- **Dashboard:** presents current conditions, recent trends, and the reason for the current state.

Decision states

The engine assigns one of three states after three consecutive samples meet the same condition.

State	Example condition	Operator expectation
Normal	All readings are inside target ranges	Continue scheduled checks
Watch	One reading is near a threshold	Inspect within 30 minutes
Act	One reading exceeds a threshold	Follow the displayed response

This is a semantic Markdown table because the content has repeated, comparable fields. Zensical renders it responsively; the export script maps it to a real Word table with a header row. Ordinary explanatory prose remains outside tables for readability and accessibility.

The three-sample rule reduces noise without hiding sustained change. Thresholds in this demo are illustrative, not horticultural advice.

Data retention

The hub stores 30 days of samples and event history. An export contains timestamps, normalized readings, state transitions, acknowledgements, and the rule-set version. It does not contain names unless an operator voluntarily enters one in a note.

Failure behavior

If one sensor becomes stale, the dashboard reports **sensor unavailable** and does not infer a replacement value. The last valid measurement remains visible with its timestamp. See Troubleshooting.

Troubleshooting

Start with the visible symptom. Preserve timestamps and exact messages when escalating a problem.

A sensor reports no data

1. Confirm the cable is fully seated and dry.
2. Check whether the dashboard timestamp continues to advance for other sensors.
3. Disconnect power, reconnect the sensor, and restore power.
4. Wait three sampling intervals.

If the sensor is still unavailable, record its port, the last valid timestamp, and the hub status indicator.

The dashboard is unavailable

Confirm that your device and the hub are on the same network. Then open `http://sentinel.local` in a new browser window. If the page still does not load, restart the hub once and wait two minutes.

A state seems incorrect

Review the three most recent samples and compare them with the decision-state rule. A state can lag a single new reading because Sentinel requires three consecutive samples.

Troubleshooting the fictional dashboard never overrides physical greenhouse safety procedures.

How this repository works

This page is the guided tour of the artifact you are reading. Every feature points to a real, inspectable implementation in the repository.

For the complete construction sequence, continue with Build it from scratch.

Author once in Markdown

All maintained content lives under `docs/`. Product guides and repository guidance use the same lightweight syntax, review process, and validation rules. Page titles use one level-one heading; sections follow a logical hierarchy; links remain relative so the content can move between renderers.

Control the information architecture

`zensical.toml` contains an explicit nav list. This makes the intended order and grouping reviewable instead of relying on a renderer to infer structure from filenames. The validation script also fails when a Markdown page is missing from navigation.

Keep cross-references stable

High-value targets use deliberate HTML anchors such as `decision-states` in the architecture guide. `scripts/validate_docs.py` resolves every internal Markdown link and fragment before publication. Changing a heading can no longer silently strand an important link.

The product pages include labeled **Feature in action** notes beside representative examples. These notes expose the implementation without interrupting the primary task for readers who only need the fictional product instructions.

See each authoring feature work

- **Cross-reference:** the operating guide links to `architecture.md#decision-states`; the explicit anchor is stable and validated.
- **Descriptive link:** the quick start names the troubleshooting symptom, which stays meaningful outside its surrounding sentence.
- **Semantic table:** the architecture page uses a table only for repeated state, condition, and expectation fields; exports preserve the header relationship.
- **Admonition:** `!!! info` creates a visually distinct note while retaining a text label and readable source.
- **Code block:** fenced blocks preserve commands and sample output; the site adds copy support and the Word export applies a monospaced style.
- **Explicit navigation:** `zensical.toml` controls order and labels; the validator detects both missing targets and unlisted pages.
- **Brand tokens:** CSS variables change presentation globally while Markdown meaning and portable outputs remain intact.
- **Multi-format publishing:** the export script reads the same navigation order as the site, so the handbook is not a separately maintained copy.

Separate content from presentation

`docs/assets/stylesheets/brand.css` defines brand tokens and a small set of presentation rules. `docs/assets/images/mark.svg` contains the fictional mark. A new organization can replace the visual layer without editing product meaning. Try the branding exercise.

Validate before rendering

`scripts/validate_docs.py` checks:

- every configured navigation target exists;
- every maintained Markdown page appears in navigation;
- internal files and anchors resolve;
- each page has exactly one level-one heading;
- images have useful alternative text; and
- common inaccessible link labels are rejected.

`zensical build --strict` requests the renderer's strict behavior. Zensical 0.0.23 currently reports that strict mode is unsupported, so the custom validator is the enforceable gate today. It is deliberately independent of the renderer so core quality checks remain if the publishing engine changes.

Build and review automatically

`.github/workflows/docs.yml` runs validation, builds the Zensical site in strict mode, generates Word output, converts it to PDF, performs basic artifact checks, and uploads the results for review. Pull requests get the same gates as the default branch.

Publish several channels

Zensical creates the searchable HTML experience. `scripts/export_handbook.py` assembles the same maintained Markdown into a styled Word handbook. CI converts that Word file to PDF. These are publishing transformations, not separate authoring silos.

Treat accessibility as an authoring constraint

The accessibility standard covers heading order, descriptive links, alternative text, tables, keyboard focus, zoom, contrast, and reduced motion. CSS enhances focus visibility and avoids essential animation.

Release with evidence

The release checklist joins automated checks with human review: wording, task completeness, responsive behavior, keyboard use, zoom, and generated-file inspection.

Build it from scratch

This chapter records every step used to construct this repository. Follow it to reproduce the demo or adapt the workflow for another small documentation set.

1. Define the public boundary

Choose a fictional product with enough behavior for procedures, concepts, reference material, troubleshooting, and cross-references. Record the release rule before drafting: no employer, customer, proprietary, export-controlled, or classified information. Greenhouse Sentinel is intentionally fictional and recommends—but never performs—physical actions.

2. Create an independent repository

Use a dedicated repository so its dependencies and automation run only when this project changes.

```
.
├── .github/workflows/docs.yml
├── deliverables/
├── docs/
│   ├── about/
│   ├── assets/
│   ├── contributing/
│   └── product/
├── scripts/
├── README.md
├── requirements.txt
└── zensical.toml
```

Ignore `site/`, virtual environments, caches, and local QA files. Keep portfolio-ready files in `deliverables/`.

3. Design the information architecture

Create three reader journeys: **Product guide** for realistic technical content, **How this repository works** for implementation details, and **Contribute** for authoring and quality standards. Write their order and labels explicitly in the nav array in `zensical.toml`.

4. Configure the renderer

Pin Zensical in `requirements.txt`. Set the site identity, public and repository URLs, navigation, theme features, light and dark palettes, Markdown extensions, and custom stylesheet in `zensical.toml`. Use links to Markdown files so source previews and renderers can both resolve them.

5. Write a thin but complete product set

Create the product pages in reader order:

1. `quickstart.md` establishes the first successful outcome.

2. `operations.md` documents a repeatable task.
3. `architecture.md` explains boundaries, components, rules, and retention.
4. `troubleshooting.md` begins with observable symptoms and preserves escalation evidence.

Use one level-one heading per page, sequential subheadings, direct instructions, expected results, and honest fictional-product limitations.

6. Add stable cross-references

Add deliberate lowercase anchors before high-value targets:

```
<a id="decision-states"></a>
## Decision states
```

Link with a relative source path and stable fragment:

```
[Decision states](architecture.md#decision-states)
```

This decouples important links from incidental heading punctuation.

Explain the feature where readers encounter it. This demo uses a labeled `!!! info` note after the representative cross-reference. The note identifies the source syntax, published behavior, accessibility benefit, and validation rule. Apply the same pattern selectively to tables, code blocks, admonitions, navigation, branding, and exports so the repository demonstrates rather than merely claims each capability.

7. Build independent validation

Create `scripts/validate_docs.py` with Python standard-library dependencies. Read `zensical.toml`, flatten navigation, discover maintained Markdown, extract headings and anchors, and resolve every local link relative to its source.

Fail when navigation names a missing file, a maintained page is omitted, a file or anchor does not resolve, a page lacks exactly one level-one heading, heading levels skip, an informative image has no alternative text, or a link uses context-free wording such as “click here.” Keep this separate from Zensical so renderer changes do not weaken the release gate.

8. Add replaceable branding

Create `docs/assets/stylesheets/brand.css` with purpose-based tokens for primary, accent, surface, text, focus, and radius values. Add a compact accessible SVG mark. Use the tokens for visible focus, mobile button stacking, readable line lengths, and reduced-motion behavior. Do not place brand decisions in Markdown.

9. Generate Word from reviewed source

Create `scripts/export_handbook.py`. Read the same navigation list as the site and process pages in that order. Map Markdown headings to real Word heading styles, sequences and bullets to list styles, code to a monospaced style, and Markdown tables to Word tables. Apply explicit page geometry, typography, spacing, colors, footer text, and public document properties.

10. Generate PDF

In CI, convert the Word handbook with headless LibreOffice. Check that Word and PDF files exist and are non-empty. Render every page to images for visual review; inspect for clipping, overlap, broken tables, awkward breaks, missing glyphs, headers, and footers.

11. Automate the workflow

Create `.github/workflows/docs.yml` for pushes, pull requests, and manual runs. The job checks out the repo, installs pinned dependencies, runs independent validation, requests a strict Zensical build, generates Word and PDF, checks artifacts, and uploads the site plus portable files for review.

Zensical 0.0.23 currently warns that strict mode is unsupported. Retain the flag for forward compatibility, but make the independent validator the enforceable gate.

12. Test the site

Build and serve `site/`. Test at 320, 360, 390, 768, 1024, and 1440 CSS pixels. At every width, verify `scrollWidth <= clientWidth`. Test the skip link, navigation, search, theme control, links, buttons, visible focus, heading order, descriptive links, 200% zoom, long-label wrapping, touch targets, light and dark contrast, and reduced motion.

13. Review for public release

Complete the release checklist. Inspect source history, rendered pages, generated files, document properties, repository description, and README for sensitive or employer-specific material. Confirm every limitation is stated plainly.

14. Publish the GitHub repository

Initialize Git in this folder, set `main` as the default branch, commit the verified files, create a public repository named `greenhouse-sentinel-docs`, and push only this folder. Require the documentation workflow before merge when account settings allow it. Optionally deploy `site/` with GitHub Pages.

15. Link from the portfolio

After publication, add a small portfolio entry that links to this repository and optionally its live site. Describe the reader problem and demonstrated capabilities. Do not embed this repository's build into the portfolio build.

16. Update and release

For every change: edit the isolated source or presentation layer, add new pages to navigation, validate, build, regenerate portable outputs, complete proportional editorial/accessibility/responsive QA, open a focused pull request, and merge only after CI and review pass.

Branding

Branding belongs in the publishing layer. The source should still make sense as plain Markdown, in GitHub preview, in Word, and in a different web theme.

Five-minute rebrand

1. Open `docs/assets/stylesheets/brand.css`.
2. Replace the values under `:root` for the primary, accent, surface, text, and focus colors.
3. Replace `docs/assets/images/mark.svg` with a square logo that has a meaningful filename.
4. Update `site_name`, `site_author`, and `copyright` in `zensical.toml`.
5. Run the validator and a strict production build.
6. Test the rendered site at the widths in the release checklist.

Token approach

The stylesheet maps brand decisions to purpose-based tokens such as `--brand-primary` and `--brand-focus`. Components use those tokens rather than repeating literal colors. This keeps a rebrand small, reviewable, and consistent.

Content that should not be branded

Do not embed organization names in reusable technical explanations, hard-code colors into diagrams without a legend, or use a logo as the only way to identify a page. Brand changes should not alter instructions, safety meaning, heading order, or link destinations.

Publishing

The repository treats publishing as a reproducible transformation of reviewed source.

HTML with Zensical

Zensical is the preferred renderer because it supports explicit navigation, a searchable static site, and a modern responsive theme. The checked-in configuration keeps the local preview and CI build aligned.

```
zensical build --strict
```

Version 0.0.23 currently warns that strict mode is unsupported. The command is retained for forward compatibility, but the independent documentation validator—not that flag—is the required link, anchor, structure, and navigation gate.

Word and PDF

The export script reads the explicit navigation list so portable outputs follow the same order as the site. It applies a restrained document style, real heading levels, a running footer, and link text that remains understandable outside the site.

```
python scripts/export_handbook.py
```

CI uses LibreOffice to convert the generated Word file to PDF. Both files are checked for existence and plausible size before they are uploaded as build artifacts.

Renderer-independent gates

A replacement renderer is acceptable only if the release continues to verify navigation completeness, target files, anchors, accessible structure, responsive layout, and portable outputs. Tool choice can evolve; the acceptance criteria stay explicit.

Writing guide

Write for the person completing a task, then make the implementation easy to review.

Structure

- Use one level-one heading per page.
- Keep heading levels sequential; do not skip from level two to level four.
- Begin procedures with the reader's goal and prerequisites.
- Use numbered lists for sequences and bullets for unordered choices.
- Put reusable, high-value targets behind deliberate lowercase anchors.

Language

- Prefer direct verbs: **Select Save**, not **The Save button should be selected**.
- State expected results after important steps.
- Use consistent terms for components and states.
- Avoid “click here,” “read more,” and other links that lose meaning out of context.
- Label fictional values and limitations honestly.

Cross-references

Use relative Markdown paths and include the stable fragment when linking to a section:

```
[Decision states](../product/architecture.md#decision-states)
```

Run `python scripts/validate_docs.py` before opening a pull request.

Accessibility

Accessibility is part of authoring and release, not a cleanup pass after publication.

Authoring standard

- Use a logical heading hierarchy and one clear page title.
- Write descriptive link text that makes sense in a list of links.
- Add concise alternative text to informative images; hide decorative imagery from assistive technology in templates.
- Give tables a header row and use tables only for genuinely tabular relationships.
- Do not use color as the only way to communicate state.
- Keep instructions independent of visual position such as “the green button on the right.”

Rendered-site checks

- Navigate every interactive control with a keyboard and confirm focus is visible.
- Verify the skip link reaches the main content.
- Confirm text and controls remain usable at 200% browser zoom.
- Test at 320, 360, 390, 768, 1024, and 1440 CSS pixels.
- At every width, confirm the page has no horizontal overflow.
- Respect `prefers-reduced-motion`; do not require animation to understand content.

Portable-output checks

Inspect every generated Word and PDF page for clipped text, broken tables, poor page breaks, missing glyphs, and unclear link text. Confirm Word uses real heading styles so readers can navigate by structure.

Release checklist

Automated gates

- [ ] The independent documentation validator passes.
- [ ] `zensical build --strict` succeeds.
- [ ] Every maintained page is present in explicit navigation.
- [ ] Internal files and anchors resolve.
- [ ] Word and PDF outputs are generated and pass artifact checks.

Editorial review

- [ ] The change answers a reader goal and states expected results.
- [ ] Terms, labels, units, and cross-references are consistent.
- [ ] No confidential, customer, proprietary, export-controlled, or classified information is present.
- [ ] Fictional values and limitations are identified.

Accessibility and responsive review

- [ ] Heading order and landmark structure are logical.
- [ ] Keyboard focus is visible and the skip link works.
- [ ] Text remains usable at 200% zoom.
- [ ] Pages have no horizontal overflow at 320, 360, 390, 768, 1024, and 1440 CSS pixels.
- [ ] Long labels, buttons, navigation, email addresses, and graphics wrap or crop safely.
- [ ] Motion is reduced when the operating-system preference requests it.

Generated-file review

- [ ] Every Word and PDF page has been visually inspected.
- [ ] Headings, lists, tables, links, headers, and footers render correctly.
- [ ] File names, document properties, and visible content are suitable for public release.